

How to Build an AI From Scratch: A Complete Step-by-Step Guide

Ardit · Grow Labs, Amaro Than Platforms · 22 September 2026

Abstract

This paper is a practical guide to building a small language model from zero: the tools to install, where to get data, how to turn text into numbers, the full model code, how to train it, which hyperparameters to use, and how to make it write real sentences instead of gibberish. All code is included and runs on a laptop (Apple M1/M2/M3 with MPS, an NVIDIA GPU with CUDA, or a normal CPU). The main lesson is simple: the quality of the text a model writes depends mostly on the amount and quality of data, how long it trains, and how you sample from it.

1. The big picture

A language model learns one thing: **given some text, predict the next token**. A token is a word or a piece of a word. If it does this well millions of times, it can write whole paragraphs.

Step	What you do	File
1. Setup	Install Python and libraries	-
2. Data	Collect and clean text	prepare.py
3. Tokenize	Turn text into numbers	prepare.py
4. Model	Build a small transformer	model.py
5. Train	Show it data and correct its mistakes	train.py
6. Generate	Make it write, using temperature and sampling	generate.py

2. Step 1: Setup

You need Python 3.10 or newer. Create a folder, a virtual environment, and install four libraries:

```
mkdir tinygpt && cd tinygpt
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install torch numpy tiktoken datasets
```

torch (PyTorch) builds and trains the model. **numpy** handles arrays of numbers. **tiktoken** is the GPT-2 tokenizer. **datasets** downloads data from Hugging Face. On an Apple Silicon Mac, PyTorch uses the GPU automatically through MPS.

3. Step 2: Getting data

Data is the most important part. **A model can only write as well as the text it learned from.**

Source	What it is	Good for
roneneldan/TinyStories	About 2 million short, simple children's stories written by AI	Best first dataset: small models learn to write clean English fast
wikitext-103	About 100 million tokens of Wikipedia	Facts and formal writing
HuggingFaceFW/fineweb-edu	Filtered educational web pages (very large)	General knowledge; take a slice

Source	What it is	Good for
Project Gutenberg	Free public-domain books	Long, literary text
Your own JSONL	Prompt and response pairs you write or collect	Teaching a specific style, topic or language

Your own data in JSONL. One JSON object per line:

```
{"prompt": "What is overfitting?", "response": "Overfitting is when a model memorises..."}
{"prompt": "What is a token?", "response": "A token is a word or part of a word..."}
```

Cleaning rules: remove empty and very short pieces, remove duplicates, remove broken characters and HTML, and keep one language or clearly mark each language. Bad data teaches bad output.

4. Step 3: Tokenization

Computers only understand numbers. A **tokenizer** splits text into pieces and gives each an ID.

Tokenizer	Example: "learning"	Vocabulary	When to use
Character	l, e, a, r, n, i, n, g (8 tokens)	About 100	Very small data, or a language GPT-2 handles badly
GPT-2 BPE (tiktoken)	learn, ing (2 tokens)	50,257	Most English projects; faster learning

The script below loads data (a text file, your JSONL, or a Hugging Face dataset), cleans it, tokenizes it, and splits it into 90% training and 10% validation data. The validation part is hidden from training so we can check if the model really learned.

File: prepare.py

```
# prepare.py - Step 1: turn raw text into token IDs for training.
# Usage:
# python prepare.py --input data.txt                (a plain text file)
# python prepare.py --input data.jsonl             (JSONL with "prompt"/"response" or "text")
# python prepare.py --hf roneneldan/TinyStories     (download a Hugging Face dataset)
# Add --tokenizer char to use characters instead of GPT-2 tokens (no download needed).
import argparse, json, os, pickle
import numpy as np

p = argparse.ArgumentParser()
p.add_argument("--input", help="path to .txt or .jsonl")
p.add_argument("--hf", help="Hugging Face dataset name, e.g. roneneldan/TinyStories")
p.add_argument("--hf_rows", type=int, default=200_000, help="how many rows to take from the HF dataset")
p.add_argument("--tokenizer", choices=["gpt2", "char"], default="gpt2")
p.add_argument("--out", default="data")
args = p.parse_args()

# ---- 1. Load text ----
texts = []
if args.hf:
    from datasets import load_dataset          # pip install datasets
    ds = load_dataset(args.hf, split="train")
    col = "text" if "text" in ds.column_names else ds.column_names[0]
    texts = [row[col] for row in ds.select(range(min(args.hf_rows, len(ds))))]
elif args.input.endswith(".jsonl"):
    with open(args.input, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            obj = json.loads(line)
            if "text" in obj:
                texts.append(obj["text"])
            else:
                texts.append(f"User: {obj['prompt']}\nAssistant: {obj['response']}")
```

```

else:
    with open(args.input, encoding="utf-8") as f:
        texts = f.read().split("\n\n")

# ---- 2. Clean ----
texts = [t.strip() for t in texts if len(t.strip()) > 20] # drop empty / tiny pieces
texts = list(dict.fromkeys(texts)) # drop exact duplicates
print(f"documents after cleaning: {len(texts):,}")

# ---- 3. Tokenize ----
os.makedirs(args.out, exist_ok=True)
if args.tokenizer == "gpt2":
    import tiktoken # pip install tiktoken
    enc = tiktoken.get_encoding("gpt2")
    eot = enc.eot_token # marks the end of each document
    ids = []
    for t in texts:
        ids.extend(enc.encode_ordinary(t)); ids.append(eot)
    meta = {"tokenizer": "gpt2", "vocab_size": enc.n_vocab}
else:
    joined = "\n\n".join(texts)
    chars = sorted(set(joined))
    stoi = {c: i for i, c in enumerate(chars)}
    ids = [stoi[c] for c in joined]
    meta = {"tokenizer": "char", "vocab_size": len(chars), "chars": chars}

ids = np.array(ids, dtype=np.uint32)
print(f"total tokens: {len(ids):,} vocab size: {meta['vocab_size']}")

# ---- 4. Split: 90% train, 10% validation ----
n = int(len(ids) * 0.9)
ids[:n].tofile(os.path.join(args.out, "train.bin"))
ids[n:].tofile(os.path.join(args.out, "val.bin"))
with open(os.path.join(args.out, "meta.pkl"), "wb") as f:
    pickle.dump(meta, f)
print("saved to", args.out)

```

Run it:

```

python prepare.py --hf roneneldan/TinyStories --hf_rows 200000 # recommended first run
python prepare.py --input mydata.jsonl # your own data
python prepare.py --input mybook.txt --tokenizer char # character level

```

5. Step 4: The model

We build a small **transformer**, the same design as GPT and Claude, only much smaller. It has:

- **Token embedding:** turns each token ID into a list of numbers (a vector).
- **Position embedding:** tells the model where each token is in the sentence.
- **Transformer blocks:** each has *attention* (each token looks at earlier tokens and decides which matter) and an *MLP* (processes what it found).
- **Output head:** gives a score for every possible next token.

File: model.py

```

# model.py - Step 2: a small GPT-style transformer.
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class Block(nn.Module):
    """One transformer layer: attention (look at earlier tokens) + MLP (think about it)."""
    def __init__(self, d_model, n_heads, dropout):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.proj = nn.Linear(d_model, d_model)

```

```

self.ln2 = nn.LayerNorm(d_model)
self.mlp = nn.Sequential(
    nn.Linear(d_model, 4 * d_model), nn.GELU(),
    nn.Linear(4 * d_model, d_model), nn.Dropout(dropout))
self.n_heads, self.dropout = n_heads, dropout
self.resid_drop = nn.Dropout(dropout)

def forward(self, x):
    B, T, C = x.shape
    q, k, v = self.qkv(self.ln1(x)).split(C, dim=2)
    q, k, v = [t.view(B, T, self.n_heads, C // self.n_heads).transpose(1, 2) for t in (q, k, v)]
    # causal=True: each token can only look at tokens BEFORE it
    a = F.scaled_dot_product_attention(q, k, v, is_causal=True,
                                       dropout_p=self.dropout if self.training else 0.0)
    a = a.transpose(1, 2).contiguous().view(B, T, C)
    x = x + self.resid_drop(self.proj(a))
    x = x + self.mlp(self.ln2(x))
    return x

class TinyGPT(nn.Module):
    def __init__(self, vocab_size, block_size, d_model, n_layers, n_heads, dropout):
        super().__init__()
        self.block_size = block_size
        self.tok_emb = nn.Embedding(vocab_size, d_model) # token ID -> vector
        self.pos_emb = nn.Embedding(block_size, d_model) # position -> vector
        self.drop = nn.Dropout(dropout)
        self.blocks = nn.ModuleList([Block(d_model, n_heads, dropout) for _ in range(n_layers)])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False) # vector -> score for every token
        self.head.weight = self.tok_emb.weight # weight tying: saves parameters
        self.apply(self._init)

    def _init(self, m):
        if isinstance(m, (nn.Linear, nn.Embedding)):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)
        if isinstance(m, nn.Linear) and m.bias is not None:
            nn.init.zeros_(m.bias)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        pos = torch.arange(T, device=idx.device)
        x = self.drop(self.tok_emb(idx) + self.pos_emb(pos))
        for blk in self.blocks:
            x = blk(x)
        logits = self.head(self.ln_f(x))
        loss = None
        if targets is not None: # cross-entropy: how surprised the model is by the real next token
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=0.8, top_k=50, top_p=0.95,
                repetition_penalty=1.1):
        for _ in range(max_new_tokens):
            ctx = idx[:, -self.block_size:]
            logits, _ = self(ctx)
            logits = logits[:, -1, :]
            # 1) repetition penalty: make tokens we already used a bit less likely
            if repetition_penalty != 1.0:
                prev = torch.unique(ctx)
                sel = logits[:, prev]
                logits[:, prev] = torch.where(sel > 0, sel / repetition_penalty, sel * repetition_penalty)
            # 2) temperature: <1 = safer and more focused, >1 = more random
            logits = logits / max(temperature, 1e-5)
            # 3) top-k: only keep the k most likely tokens
            if top_k:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                logits[logits < v[:, [-1]]] = -float("inf")
            probs = F.softmax(logits, dim=-1)
            # 4) top-p (nucleus): keep the smallest set of tokens whose probability adds up to p
            if top_p < 1.0:
                sp, si = torch.sort(probs, descending=True)
                cum = torch.cumsum(sp, dim=-1)
                sp[(cum - sp) > top_p] = 0
                probs = torch.zeros_like(probs).scatter(-1, si, sp)

```

```

        probs = probs / probs.sum(dim=-1, keepdim=True)
        nxt = torch.multinomial(probs, 1)
        idx = torch.cat([idx, nxt], dim=1)
    return idx

```

```

def count_params(model):
    return sum(p.numel() for p in model.parameters())

```

6. Step 5: Training

Training repeats four steps thousands of times: take a random batch of text, predict the next tokens, measure the error (**loss**), and adjust the weights to reduce it. The script also tracks the **gap** between validation and training loss, and saves the model only when validation loss improves.

File: train.py

```

# train.py - Step 3: train the model.
# Usage: python train.py --size small          (tiny | small | medium)
import argparse, math, os, pickle, time
import numpy as np
import torch
from model import TinyGPT, count_params

SIZES = { #          layers heads d_model block batch lr      dropout
    "tiny": dict(n_layers=4, n_heads=4, d_model=256, block_size=256, batch_size=32, lr=6e-4, dropout=0.1),
    "small": dict(n_layers=6, n_heads=6, d_model=384, block_size=256, batch_size=32, lr=5e-4, dropout=0.1),
    "medium": dict(n_layers=8, n_heads=8, d_model=512, block_size=512, batch_size=16, lr=3e-4, dropout=0.1),
}

p = argparse.ArgumentParser()
p.add_argument("--size", default="small", choices=SIZES)
p.add_argument("--data", default="data")
p.add_argument("--max_steps", type=int, default=20000)
p.add_argument("--eval_every", type=int, default=500)
p.add_argument("--weight_decay", type=float, default=0.1)
p.add_argument("--out", default="out")
args = p.parse_args()
cfg = SIZES[args.size]

device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
torch.manual_seed(1337)
print("device:", device)

meta = pickle.load(open(os.path.join(args.data, "meta.pkl"), "rb"))
train = np.fromfile(os.path.join(args.data, "train.bin"), dtype=np.uint32)
val = np.fromfile(os.path.join(args.data, "val.bin"), dtype=np.uint32)

def get_batch(split):
    d = train if split == "train" else val
    ix = np.random.randint(0, len(d) - cfg["block_size"] - 1, cfg["batch_size"])
    x = torch.stack([torch.from_numpy(d[i:i + cfg["block_size"]]).astype(np.int64) for i in ix])
    y = torch.stack([torch.from_numpy(d[i + 1:i + 1 + cfg["block_size"]]).astype(np.int64) for i in ix])
    return x.to(device), y.to(device)

model = TinyGPT(meta["vocab_size"], cfg["block_size"], cfg["d_model"],
                cfg["n_layers"], cfg["n_heads"], cfg["dropout"]).to(device)
n_params = count_params(model)
print(f"parameters: {n_params/1e6:.1f}M   training tokens: {len(train)/1e6:.1f}M   "
      f"tokens per parameter: {len(train)/n_params:.1f}")

opt = torch.optim.AdamW(model.parameters(), lr=cfg["lr"], betas=(0.9, 0.95),
                        weight_decay=args.weight_decay)
warmup = max(100, args.max_steps // 50)

def lr_at(step): # warm up, then slowly lower the learning rate (cosine schedule)
    if step < warmup:
        return cfg["lr"] * (step + 1) / warmup
    progress = (step - warmup) / max(1, args.max_steps - warmup)
    return cfg["lr"] * (0.1 + 0.9 * 0.5 * (1 + math.cos(math.pi * progress)))

@torch.no_grad()
def evaluate(iters=50):

```

```

model.eval()
out = {}
for split in ("train", "val"):
    out[split] = sum(model(*get_batch(split))[1].item() for _ in range(iters)) / iters
model.train()
return out

os.makedirs(args.out, exist_ok=True)
best_val, t0 = float("inf"), time.time()
for step in range(args.max_steps + 1):
    for g in opt.param_groups:
        g["lr"] = lr_at(step)
    if step % args.eval_every == 0:
        l = evaluate()
        gap = l["val"] - l["train"]
        print(f"step {step:6d} | train {l['train']:.3f} | val {l['val']:.3f} | gap {gap:.3f} "
              f"| lr {lr_at(step):.1e} | {time.time()-t0:.0f}s")
        if l["val"] < best_val: # only keep the checkpoint with the best VALIDATION loss
            best_val = l["val"]
            torch.save({"model": model.state_dict(), "cfg": cfg, "meta": meta},
                      os.path.join(args.out, "best.pt"))
    if step == args.max_steps:
        break
    x, y = get_batch("train")
    _, loss = model(x, y)
    opt.zero_grad(set_to_none=True)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # stops sudden huge updates
    opt.step()
print(f"done. best val loss {best_val:.3f}")

```

Run it:

```
python train.py --size small --max_steps 20000
```

You will see lines like this. Loss should go down; the gap should stay small:

```
step   2000 | train 2.310 | val 2.402 | gap 0.092 | lr 4.9e-04 | 610s
```

7. Hyperparameters

Hyperparameters are the settings you choose before training. These are good starting values.

Size	Layers	Heads	d_model	Context	Batch	Learning rate	Parameters*
tiny	4	4	256	256	32	6e-4	About 16M
small	6	6	384	256	32	5e-4	About 30M
medium	8	8	512	512	16	3e-4	About 51M

* With the GPT-2 tokenizer. Most parameters are in the token embedding ($50,257 \times d_{\text{model}}$). With a character tokenizer the same models are only about 3M, 11M and 25M.

Setting	What it does	Start with	If too high	If too low
Learning rate	Size of each weight update	3e-4 to 6e-4	Loss jumps or becomes NaN; output loops	Learns very slowly
Batch size	Examples per step	16 to 64	Runs out of memory	Noisy, unstable training
Context (block size)	How many tokens the model sees at once	256 to 512	Slow, uses much memory	Forgets the start of sentences
Dropout	Randomly switches parts off to stop memorising	0.1 (0.2 for small data)	Learns slowly	Memorises (overfits)

Setting	What it does	Start with	If too high	If too low
Weight decay	Keeps weights small	0.1	Underfits	Overfits
Warmup	Starts with a low learning rate	2% of steps	Wastes steps	Unstable start
Gradient clipping	Limits sudden huge updates	1.0	-	Training explodes

How long to train. A good rule is at least **20 tokens of data for every parameter**. A 30M model wants about 600M tokens seen during training. If you have less data, use a smaller model.

8. Step 6: Generating text

After training, the model gives a probability for every possible next token. **Sampling** decides which one to pick.

Setting	What it does	Recommended
Temperature	Controls randomness. Low = safe and repetitive. High = creative but can become nonsense	0.7 to 0.9 (0.3 to 0.5 for facts)
Top-k	Only choose from the k most likely tokens	40 to 50
Top-p (nucleus)	Only choose from the smallest group of tokens whose probability adds up to p	0.9 to 0.95
Repetition penalty	Makes already-used tokens less likely, to stop loops	1.1 to 1.2

Temperature	Typical result on a small model
0.1	Repeats the same phrase over and over
0.3 to 0.5	Safe, focused, a bit boring
0.7 to 0.9	Best balance for stories and conversation
1.0 to 1.2	More creative, more mistakes
1.5 and above	Random words: gibberish

File: generate.py

```
# generate.py - Step 4: make the model write text.
# Usage: python generate.py --prompt "Once upon a time" --temperature 0.8
import argparse, pickle
import torch
from model import TinyGPT

p = argparse.ArgumentParser()
p.add_argument("--ckpt", default="out/best.pt")
p.add_argument("--prompt", default="Once upon a time")
p.add_argument("--tokens", type=int, default=200)
p.add_argument("--temperature", type=float, default=0.8)
p.add_argument("--top_k", type=int, default=50)
p.add_argument("--top_p", type=float, default=0.95)
p.add_argument("--repetition_penalty", type=float, default=1.1)
p.add_argument("--samples", type=int, default=1)
args = p.parse_args()

device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
ck = torch.load(args.ckpt, map_location=device, weights_only=False)
cfg, meta = ck["cfg"], ck["meta"]
model = TinyGPT(meta["vocab_size"], cfg["block_size"], cfg["d_model"],
                cfg["n_layers"], cfg["n_heads"], 0.0).to(device)
model.load_state_dict(ck["model"])
model.eval()
```

```

if meta["tokenizer"] == "gpt2":
    import tiktoken
    enc = tiktoken.get_encoding("gpt2")
    encode, decode = enc.encode_ordinary, enc.decode
else:
    stoi = {c: i for i, c in enumerate(meta["chars"])}
    encode = lambda s: [stoi[c] for c in s if c in stoi]
    decode = lambda ids: "".join(meta["chars"][i] for i in ids)

x = torch.tensor([encode(args.prompt)], dtype=torch.long, device=device)
for i in range(args.samples):
    y = model.generate(x, args.tokens, args.temperature, args.top_k, args.top_p, args.repetition_penalty)
    print(f"--- sample {i+1} ---")
    print(decode(y[0].tolist()))

```

Run it:

```
python generate.py --prompt "Once upon a time" --temperature 0.8 --top_k 50 --top_p 0.95
```

9. Why your model writes gibberish, and how to fix it

Gibberish is the most common problem when building a first model. It almost always has one of these causes.

Symptom	Likely cause	Fix
Random letters or broken words	Trained too little; loss still high	Train longer; loss should fall below about 2.0 (GPT-2 tokens) or 1.3 (characters)
Real words, but no sense	Not enough data for the model size	More data (aim for 20 tokens per parameter) or a smaller model
Same phrase repeated	Temperature too low, or learning rate too high	Temperature 0.7 to 0.9, repetition penalty 1.1, lower learning rate
Copies training text exactly	Overfitting: validation loss rising while training loss falls	More data, more dropout, stop earlier (use best.pt)
Makes up facts	Small models cannot store many facts	Lower temperature; more and cleaner data; bigger model
Mixes languages or strange symbols	Messy data	Clean the data; keep one language, or mark languages clearly
Loss becomes NaN or jumps	Learning rate too high	Halve the learning rate; keep gradient clipping at 1.0

9.1 More data means better generation

This is the single biggest lever. Our own results at Grow Labs showed it: doubling our dataset from 5,000 to 10,000 samples reduced the gap between validation and training loss from 0.5788 to 0.1851. More data means the model sees more patterns, memorises less and generalises more.

Training tokens	What a small model typically writes
Under 1 million	Mostly broken words and repeated fragments
About 10 million	Real words and short phrases; sentences fall apart
About 100 million	Grammatical sentences; simple stories start to make sense
500 million or more	Coherent paragraphs on the topics it learned

These are rough guides for models of 10M to 50M parameters trained on clean, simple text like TinyStories. Messy or complex data needs much more.

9.2 Quality matters as much as quantity

Simple, clean, consistent text is far easier to learn than messy web text. That is why TinyStories works so well for small models: models with only a few million parameters trained on it can write fluent, simple stories. Start simple, then add harder data.

9.3 Checklist for good text

- Clean data with no duplicates.
- At least 20 training tokens per parameter.
- Learning rate around $3e-4$ to $6e-4$, with warmup and cosine decay.
- Dropout 0.1, weight decay 0.1, gradient clipping 1.0.
- Train until validation loss stops improving; use the best checkpoint, not the last one.
- Generate with temperature 0.7 to 0.9, top-k 50, top-p 0.95, repetition penalty 1.1.
- Always read the actual output. The loss number alone can look good while the text is still poor.

10. Next steps

- **Scale up:** once text is good, grow the model and data together.
- **Instruction tuning:** train on prompt and response pairs so the model answers questions like an assistant.
- **Fine-tune an open model:** for a useful assistant quickly, fine-tuning a pre-trained model (for example with LoRA) is much faster than training from zero.
- **Evaluate properly:** test with fresh questions, including trick questions, instead of trusting one number.

11. Conclusion

Building an AI from scratch is now possible on a laptop in a few hundred lines of code. The steps are always the same: get data, tokenize it, build a transformer, train it, and sample from it carefully. The difference between gibberish and real sentences comes from three things: enough clean data, enough training with sensible hyperparameters, and good sampling settings. More data gives better generation, and reading the output is the only true test.